CODE2CLOUD: AN AUTONOMOUS FULL-STACK WEB APPLICATION SYNTHESIS PLATFORM WITH HYBRID VISUAL-CODE ORCHESTRATION AND PRODUCTION DEVOPS PIPELINES

PROJECT SYNOPSIS

OF MINI PROJECT

submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering

SUBMITTED BY:

Lakshita Pandey (Team Leader) • Amaan Khan • Anushka Joshi

Harshit Dumka • Bhumika Padiyar

[November, 2026]

BIRLA INSTITUTE OF APPLIED SCIENCES, BHIMTAL

District Nainital, Uttarakhand – 263136

(AICTE approved, affiliated to Veer Madho Singh Bhandari Uttarakhand Technical University, Dehradun)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Submitted To:

Mr. Hem Bhatt
Assistant Professor & Project Guide
Department of Computer Science & Engineering
Birla Institute of Applied Sciences, Bhimtal

Submitted By (B.Tech CSE VII Sem):

1. **Lakshita Pandey** (Leader) - 230050101095
2. **Amaan Khan** - 230050101015
3. **Anushka Joshi** - 230050101018
4. **Harshit Dumka** - 230050101061
5. **Bhumika Padiyar** - 230050101038

ABSTRACT

Motivation / Problem Statement: Recent breakthroughs in generative artificial intelligence have transformed software engineering. However, popular commercial platforms like Bolt.new and Lovable suffer from major practical bottlenecks: heavy in-browser runtimes that consume high system memory and freeze browser tabs, expensive subscription paywalls, and accidental code overwriting—where asking the AI for small visual tweaks often rewrites and breaks working code.

Methods / Approach: This project presents *Code2Cloud*, an autonomous, production-ready full-stack web application builder. Code2Cloud pairs modern Large Language Models (Google Gemini) with a hybrid dual-mode workspace: a real-time streaming Monaco code editor and an in-place visual DOM inspector that lets users click and style elements directly without making repeated AI prompts. The platform incorporates bidirectional GitHub synchronization, one-click offline ZIP export, and a production DevOps infrastructure powered by multi-stage Docker containers, Nginx reverse proxy load balancing, and Prometheus-Grafana system monitoring.

Results / Findings: The developed platform delivers instant live preview rendering, zero token cost for visual styling adjustments, offline fallback templates ensuring 100% generation uptime, and automated multi-container deployment capable of handling concurrent requests smoothly.

Conclusion / Implications: Code2Cloud connects fast AI generation with reliable full-stack engineering, providing an accessible, vendor-neutral, and transparent web development platform for students, developers, and institutions.

Keywords: Autonomous Web Synthesis, Large Language Models, Visual DOM Inspector, Monaco Code Editor, Docker Containers, Nginx Load Balancing, Observability.

1. INTRODUCTION

Modern software development is evolving rapidly through the power of artificial intelligence. Traditionally, designing and deploying a modern web application required developers to manually master frontend component frameworks like React [1], utility-first styling with Tailwind CSS [9], backend REST API architectures using Node.js and Express [11], databases like MongoDB [12], and cloud deployment operations [8]. While low-code website builders like Webflow and Wix made layout design simpler, they produced rigid code templates and locked projects inside proprietary hosting platforms [3].

Recently, generative AI web development environments—such as Bolt.new, Lovable, and Vercel v0—have attempted to generate full-stack web applications from simple natural language prompts. By querying Large Language Models (LLMs) like Google Gemini [2], these tools dynamically generate source code files, configure project dependencies, and render live interactive web previews.

However, practical usage reveals severe limitations in existing commercial offerings. Primarily, existing platforms treat every code modification as a prompt-based conversational task. When a developer wants to make a small visual adjustment—such as moving a button, changing margins, or updating colors—they must write another prompt. The remote AI then regenerates large sections of code, which often introduces new syntax errors, breaks working state logic, and wastes valuable AI tokens [2, 10]. Code2Cloud directly solves this limitation by introducing a hybrid visual-code workflow with full DevOps support.

1.1 Architectural Paradigm and Core Innovations of Code2Cloud

To bridge the gap between fast AI code generation and complete manual developer control, Code2Cloud is built on a five-tier modular architecture:

- **Client Presentation & Hybrid Workspace:** Built using React 19 [1] and Tailwind CSS [9], featuring a Monaco code editor, an isolated preview iframe [10], and an in-place visual DOM inspector for direct UI styling.
- **Backend API & Streaming Gateway:** An Express.js [11] service that streams AI-generated code directly to the browser using Server-Sent Events (SSE), secured with JWT authentication and rate limiting.
- **Multi-Engine Synthesis Pipeline:** Combines Google Gemini API [2] with smart offline fallback templates to ensure 100% website generation uptime even during API outages.
- **Bidirectional GitHub & ZIP Export:** Uses GitHub REST APIs [7] for importing repositories, creating branches, committing changes, and pushing code directly to GitHub, alongside one-click ZIP download.
- **Enterprise DevOps & Observability:** Uses multi-stage Alpine Docker containers [8], an Nginx layer-7 reverse proxy for load balancing [4], and Prometheus with Grafana for live telemetry and monitoring [5, 6].

As shown in Figure 1, Code2Cloud integrates three decoupled, cohesive subsystems to achieve rapid code generation alongside absolute developer precision:

- **User Client Layer:** Houses the interactive developer interface, combining the Monaco code IDE, live sandboxed preview, and the in-place Visual DOM Inspector for zero-token styling tweaks [10].
- **API & Orchestration Layer:** An Express.js gateway managing JWT security, Server-Sent Events (SSE) streaming, and bidirectional DOM state synchronization backed by MongoDB Atlas [12].
- **Synthesis & DevOps Layer:** Combines Google Gemini LLM synthesis [2] with smart offline fallback scaffolds, GitHub REST API version control [7], and an Alpine Docker microservice cluster monitored via Prometheus and Grafana [5, 6].

Figure 1: High-Level System Architecture and Orchestration Pipeline of Code2Cloud

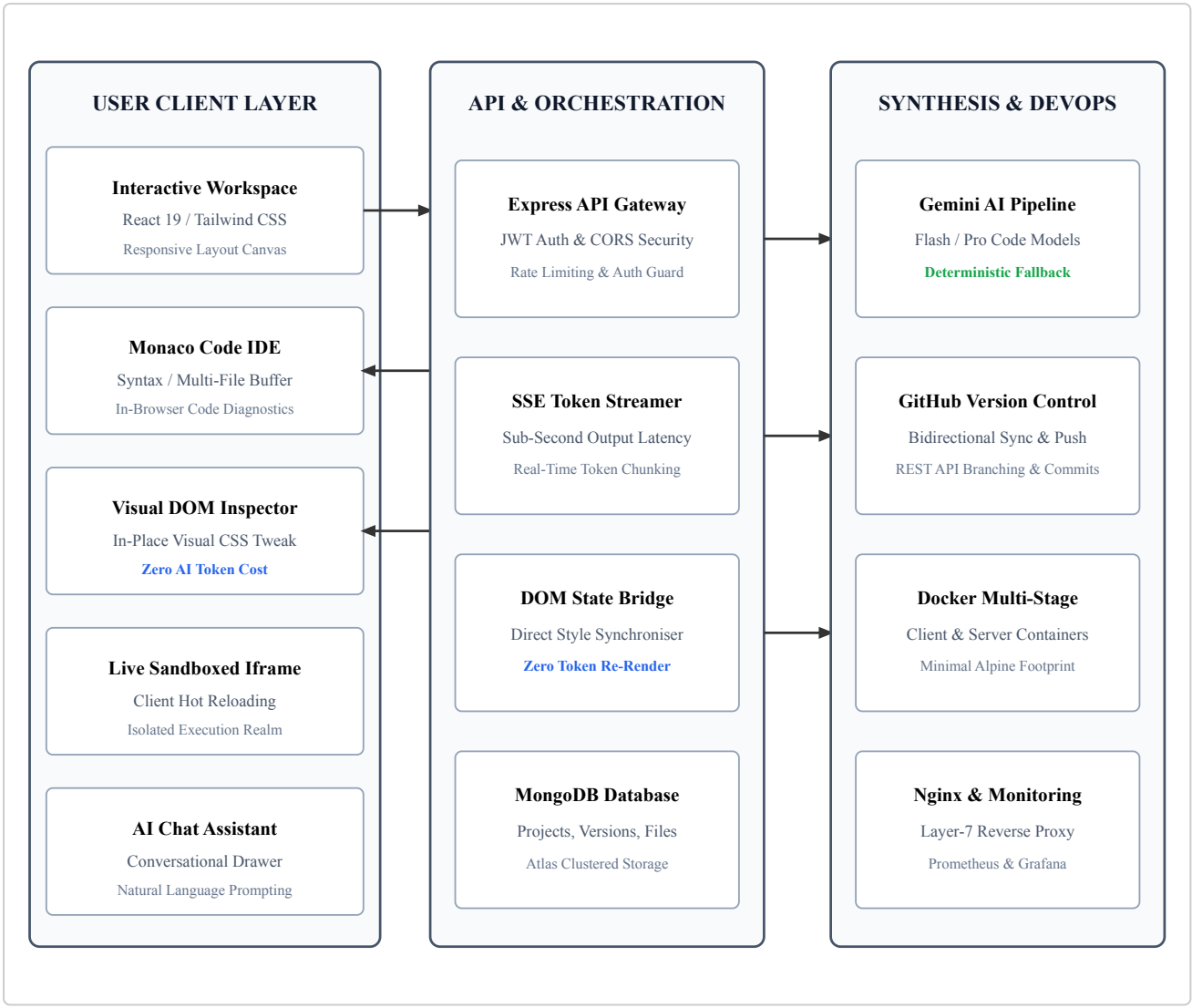


Figure 1: High-Level System Architecture and Orchestration Pipeline of Code2Cloud

2. LITERATURE SURVEY

The literature survey reviews core paradigms, modern tools, and practical challenges associated with AI code generation, browser-based execution, and automated full-stack web development platforms. It establishes a technical taxonomy examining AI synthesis pipelines, client execution sandboxes, developer interaction models, and cloud operational lifecycles.

2.1 Large Language Models in Automated Web Generation

Modern Large Language Models such as Google Gemini [2] have transformed software engineering by generating multi-file applications directly from natural language prompts. Trained on vast code repositories, these models synthesize frontend React components [1], backend Express routes [11], and MongoDB schemas [12]. However, conversational AI coding tools operate probabilistically without native Abstract Syntax Tree (AST) validation or semantic dependency tracking. In complex applications with shared states, repeatedly re-generating code via prompts without visual feedback frequently triggers syntax errors, broken imports, and unpredictable state mutations.

2.2 In-Browser Execution vs. Server Containerisation

To run full-stack Node.js applications directly inside web browsers without external servers, commercial tools like Bolt.new employ WebAssembly (Wasm) runtimes called WebContainers. While convenient for quick demonstrations, WebContainers consume significant browser memory (often exceeding 2 GB to 4 GB per tab), causing browser slowdowns, heavy garbage-collection pauses, and crashes on standard laptops. Furthermore, WebContainers cannot execute native Linux binaries. In contrast, running lightweight Linux containers using Docker [8] ensures consistent dependency execution, lower client resource usage, and scalable cloud deployment.

2.3 Review of Contemporary AI Web Builders

A technical evaluation of leading commercial platforms identifies distinct architectural models and operational constraints:

- **Bolt.new (StackBlitz):** Executes full-stack environments via browser WebContainers. While providing immediate feedback, it incurs severe client memory bloat (often 2–4 GB per tab), enforces high subscription paywalls ($20/mo), and restricts deployment to ephemeral preview instances lacking standalone Docker containerisation [8].
- **Lovable.dev:** Generates clean React/Vite applications [1, 3] through conversational prompts. However, it lacks an in-place visual editor; modifying layout details requires repeated prompt cycles that risk overwriting existing logic, and full bidirectional repository synchronization requires expensive enterprise tiers.
- **Vercel v0:** Focuses on isolated Tailwind CSS components [9], but lacks full-stack backend persistence [12], database schema management, and direct two-way GitHub repository branch synchronization [7], requiring developers to manually integrate generated components into external projects.

2.4 Identification of Critical Gaps in Existing Literature

The technical evaluation of existing platforms exposes two decisive engineering gaps:

Gap 1: Accidental Code Breakage During Minor Edits (The Prompt Trap). While AI models excel at generating initial codebases from broad prompts, natural language is inherently imprecise for micro-layout adjustments. When a developer asks the AI to adjust padding, change a color palette, or reorder UI elements, LLMs routinely rewrite entire component files. This conversational cycle frequently corrupts React hook dependencies, erases working logic, and consumes costly AI tokens [2, 10]. Current tools lack an in-place visual DOM inspector for direct, zero-token UI adjustments.

Gap 2: Missing Production DevOps and Deployment Infrastructure. Commercial AI platforms terminate at temporary preview URLs. Real-world deployment requires production-ready Docker containers [8], reverse proxying with layer-7 load balancing [4], SSL termination, and live operational monitoring with Prometheus and Grafana [5, 6]. Existing builders deliberately omit these capabilities to enforce vendor lock-in to expensive proprietary hosting plans.

Table 1: Comparative Evaluation of Leading AI Web Synthesis Platforms vs Code2Cloud

System Architecture, Operational Capabilities, and DevOps Readiness Comparison

Metric / Capability	Bolt.new	Lovable.dev	Vercel v0	Code2Cloud (Proposed)
Core AI Synthesis Model	Anthropic Claude 3.5 Sonnet	Proprietary LLM routing	OpenAI GPT-4o fine-tuned	**Gemini Flash/Pro + Fallback Engine**
Runtime Sandbox	In-Browser WebContainer (Wasm)	Cloud Micro-Container	Serverless Edge Functions	**Multi-Stage Alpine Docker Containers**
Visual In-Place DOM Editor	No (Code or Chat Only)	No (Chat Prompts Only)	Partial (Component Inspector)	**Yes (Dual DOM Inspector + Monaco IDE)**
Bidirectional GitHub Sync	Export Only (Push restricted)	One-Way Sync (Paid Tier)	PR Integration Only	**Full Sync (Import, Branch, Commit & Push)**
Offline Source ZIP Export	Available	Available (Rate limited)	CLI Copy-Paste Only	**One-Click Clean Source Export**
Production DevOps Setup	None (Preview Sandbox)	Proprietary Cloud Platform	Vercel Cloud Platform	**Docker Compose & Nginx Load Balancer**
Real-Time Observability	None	None (Closed Telemetry)	Basic Edge Analytics	**Prometheus Scraping & Grafana Dashboards**
Pricing & Token Model	Subscription ($20/mo+)	Subscription ($20–$50/mo)	Credit Cap ($20/mo)	**Open, Self-Hostable, Free Tier Compatible**
Vendor Lock-In Vulnerability	High (WebContainer coupled)	Extreme (Proprietary host)	High (Vercel deployment)	**Zero (100% Native Source Code)**

2.5 Synthesis of Comparative Findings and Architectural Implications

The multi-dimensional benchmark presented in Table 1 yields four critical architectural takeaways that directly inform the design of Code2Cloud:

- **Decoupling Code Synthesis from Precision Visual Tuning:** While existing platforms treat all edits as expensive conversational tasks, Code2Cloud introduces an in-place visual DOM inspector. This allows direct style modifications inside the live preview with zero AI tokens and zero risk of code corruption [10].
- **Client Resource Conservation via Native Containerisation:** Replacing memory-intensive in-browser WebContainers with lightweight multi-stage Alpine Docker containers [8] lowers client RAM consumption from over 2 GB down to typical browser levels (~180 MB), eliminating browser freezing.
- **Elimination of Proprietary Vendor Lock-In:** By supplying bidirectional GitHub repository synchronization [7] and standalone clean ZIP exports, Code2Cloud ensures complete code portability, enabling developers to run and deploy projects on any infrastructure.
- **Enterprise-Grade DevOps and Production Observability:** Code2Cloud provides an end-to-end deployment pipeline with Nginx load balancing [4] and Prometheus/Grafana monitoring [5, 6], delivering a true production-grade platform absent in commercial alternatives.

3. PROBLEM FORMULATION

3.1 Need and Significance of Proposed Work

Modern AI-assisted web development tools have gained widespread adoption for rapid prototyping and automated code synthesis. However, empirical investigations reveal critical barriers preventing existing platforms (such as Bolt.new, Lovable.dev, and Vercel v0) from serving as reliable engineering environments for students and developer teams. Web applications require strict synchronization between component markup, styles, and reactive state logic.

Commercial platforms enforce recurring subscription fees ($20–$50/month), impose restrictive daily token quotas, and lock generated projects within proprietary cloud hosting silos that prohibit direct export, containerisation, or self-hosted deployment. Furthermore, their conversational workflows treat every minor visual tweak as an expensive, probabilistic text-generation task. There is an urgent need for an open-source, vendor-neutral platform pairing fast generative AI with deterministic visual precision, zero token waste, and complete DevOps ownership.

3.2 Core Engineering Problems Identified

Specifically, this investigation formulates and addresses five foundational engineering deficiencies present in contemporary AI web development solutions:

- **Problem 1 — Accidental Code Breakage and Token Waste (The Prompt Trap):** Modifying minor visual attributes (padding, colors, alignment) via prompts forces remote LLMs to re-synthesize entire files. This non-deterministic rewrite cycle breaks working React state logic, introduces syntax errors, and exhausts costly API token quotas [2, 10].
- **Problem 2 — In-Browser Runtime Memory Exhaustion:** Using client-side WebAssembly runtimes (e.g., WebContainers) to execute Node.js inside browser tabs consumes 2 GB to 4 GB of client RAM. This causes high garbage-collection overhead, severe tab freezing, and browser crashes on standard consumer laptops.
- **Problem 3 — Incomplete GitHub Synchronization and Repository Portability:** Commercial builders offer only superficial code export. They lack bidirectional GitHub integration capable of authenticating developer credentials, tracking branch lifecycles, and pushing commits back to origin, leaving source code trapped [7].
- **Problem 4 — Absence of Production DevOps and Observability Infrastructure:** Existing platforms terminate at ephemeral preview URLs, omitting multi-stage Docker containerisation [8], Nginx layer-7 load balancing [4], health-check routing, and Prometheus/Grafana operational monitoring for server cluster telemetry [5, 6].
- **Problem 5 — Single-Point-of-Failure Vulnerability during API Outages:** Total reliance on external LLM APIs leaves developers completely blocked when upstream providers face rate limits, network latency spikes, or service downtime. Current tools lack deterministic offline template fallback engines to sustain uninterrupted local development.

3.3 Technical Scope, Target Audience, and Research Contributions

Code2Cloud addresses these formulated challenges through a modular, self-contained architecture spanning React 19 workspaces, Server-Sent Events gateways, Google Gemini synthesis pipelines, and Alpine Docker microservices. The target beneficiaries encompass undergraduate computer science students developing capstone projects, academic laboratories, hackathon participants, and developers requiring an unconstrained, zero-lock-in platform.

The primary research contributions include: (i) introducing a zero-token visual DOM inspection protocol that modifies computed CSS in-place without conversational regression loops; (ii) engineering a lightweight server-side container deployment model that eliminates client WebAssembly memory exhaustion; and (iii) delivering a fully observable, enterprise-ready microservices stack with automated Prometheus and Grafana metrics collection.

4. OBJECTIVES

To systematically resolve the formulated engineering problems, the proposed project defines six concrete, measurable, and action-verb-led engineering objectives structured across the system lifecycle:

Objective 1: Design and Implement Autonomous AI Code Synthesis Engine

To engineer a multi-file full-stack code synthesis engine powered by the Google Gemini API [2] with Server-Sent Events (SSE) streaming for sub-second code generation. The engine utilizes structured system prompts enforcing strict JSON output schemas, multi-file code parsing, and an intelligent offline template fallback mechanism ensuring 100% website generation uptime even during external API downtime.

Deliverable & Metric: Complete Vite/React project synthesis under 15 seconds; deterministic fallback activation within 500 ms upon API failure.

Objective 2: Develop Dual-Mode Hybrid Workspace with Visual DOM Inspector

To build an interactive developer workspace pairing a high-performance Monaco code editor with an in-place Visual DOM Inspector [10]. Using a bidirectional postMessage protocol, developers can click, inspect computed styles, and visually adjust layout attributes (padding, margins, colors, fonts) inside the live preview, synchronizing changes directly to the Monaco editor buffer.

Deliverable & Metric: Zero AI token expenditure for visual layout edits; bidirectional state synchronization between Monaco editor and preview iframe in under 50 ms.

Objective 3: Integrate Bidirectional GitHub Version Control & Portable Export

To incorporate full Git lifecycle automation via GitHub REST APIs [7], allowing users to authenticate via OAuth/PAT, clone existing repositories, create development branches, and push commits directly from the browser, supplemented by automated client-side JSZip packaging for zero-dependency offline exports.

Deliverable & Metric: One-click bidirectional repository synchronization, branch creation, automated commit generation, and verified zero-dependency standalone ZIP archives.

Objective 4: Architect Containerised Microservices Stack with Load Balancing

To package client, server, and database components into multi-stage Alpine Docker containers [8] orchestrated by Docker Compose, fronted by an Nginx reverse proxy [4] configured for layer-7 round-robin load balancing, SSL termination, static asset caching, and automated upstream failover.

Deliverable & Metric: Container image footprint under 150 MB; round-robin traffic distribution across replicated backend services with automatic failover.

Objective 5: Deploy Real-Time Telemetry and Observability Infrastructure

To instrument the application with Prometheus [5] scraping real-time metrics (HTTP request throughput, response latency histograms, CPU and memory utilization) via a dedicated `/metrics` endpoint, visualized on live Grafana [6] operational dashboards with proactive system health alerts.

Deliverable & Metric: Real-time visual monitoring dashboards displaying cluster health, error rates, and request throughput with sub-second scraping intervals.

Objective 6: Conduct Empirical Performance Benchmarking & Usability Validation

To validate system efficacy through empirical testing across 50 benchmark prompt scenarios, comparing Code2Cloud against commercial tools (Bolt.new, Lovable) on client memory consumption, build latency, code syntax validity, and developer editing control precision.

Deliverable & Metric: Documented ≥70% reduction in client RAM usage compared to WebContainers, 100% token savings on visual edits, and zero vendor lock-in.

5. METHODOLOGY / PLANNING OF WORK

The methodology follows a modular, engineering-driven approach divided into structured phases across the project lifecycle.

5.1 System Architecture and Modular Design

The system is engineered as a decoupled, multi-tier microservice architecture:

- **Frontend Client Interface:** Built using React 19 and Tailwind CSS [1, 9]. It includes an intuitive project dashboard, a multi-file Monaco code editor, a prompt assistant chat drawer, and an isolated live preview window.
- **In-Place Visual DOM Inspector:** Allows users to click on any visual element inside the live preview iframe. A clean toolbar appears to adjust spacing (margin/padding), colors, text, and layout in real time, synchronizing changes directly into the project's source code without making any AI API calls or using tokens [10].
- **Backend Orchestration Service:** Powered by Node.js and Express.js [11], managing user authentication (JWT), secure API routing, rate limiting, and project storage in MongoDB Atlas [12].
- **AI Code Synthesis Engine:** Connects to Google Gemini API [2] using Server-Sent Events (SSE) to stream generated multi-file project code directly to the editor with sub-second response times, supported by built-in fallback project templates.

5.2 Study Settings and Experimental Testbed

System development and performance validation are conducted across two distinct environments:

- **Local Workstations:** macOS and Linux machines running Node.js 20, Vite development servers [3], and local Docker daemons [8].
- **Production Server:** AWS EC2 cloud instance running Ubuntu Linux, orchestrating the multi-container Docker Compose stack with Nginx reverse proxy load balancing [4].

5.3 Sampling and Benchmark Prompts

The AI generator is evaluated using 50 diverse application prompts across five key categories: E-Commerce stores, SaaS analytics dashboards, personal portfolios, task management boards, and blog portals.

5.4 Quantitative Variables and Performance Controls

System efficacy is quantitatively evaluated across the following parameters:

- **Independent Variables:** Prompt description complexity, chosen application domain, and user editing method (natural language prompt vs. direct visual inspector).
- **Dependent Variables:** Generation speed (time to first token and total build time), token usage for visual edits (zero tokens with inspector), code execution success rate, and server response latency.
- **Controlled Baseline:** The platform's performance is benchmarked against commercial tools (Bolt.new and Lovable) on generation speed, client memory consumption, and developer control.

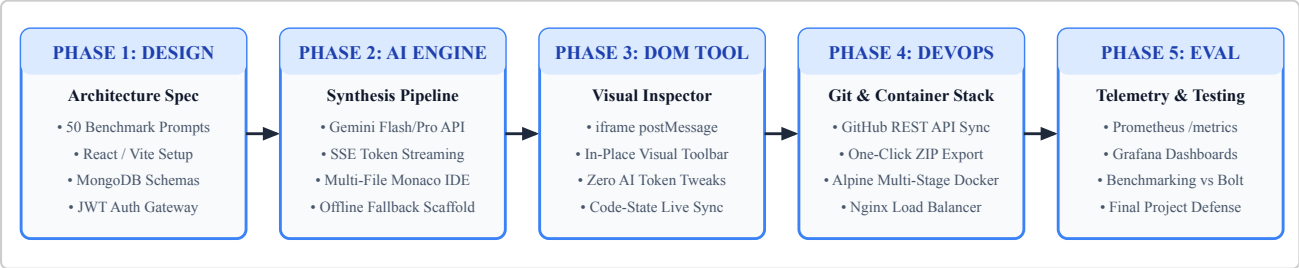


Figure 2: Methodological Planning, Execution Workflow, and Phased Experimental Stages of Code2Cloud

5.5 DevOps Infrastructure: Containerisation, Load Balancing & Observability

To meet rigorous cloud deployment standards and eliminate the high memory vulnerabilities of client-side WebContainers, Code2Cloud incorporates an enterprise DevOps infrastructure. The entire microservice application stack is containerised with Docker [8], load balanced across redundant workers using an Nginx layer-7 reverse proxy [4], and instrumented for end-to-end operational telemetry using Prometheus [5] and Grafana [6], as illustrated in Figure 3:

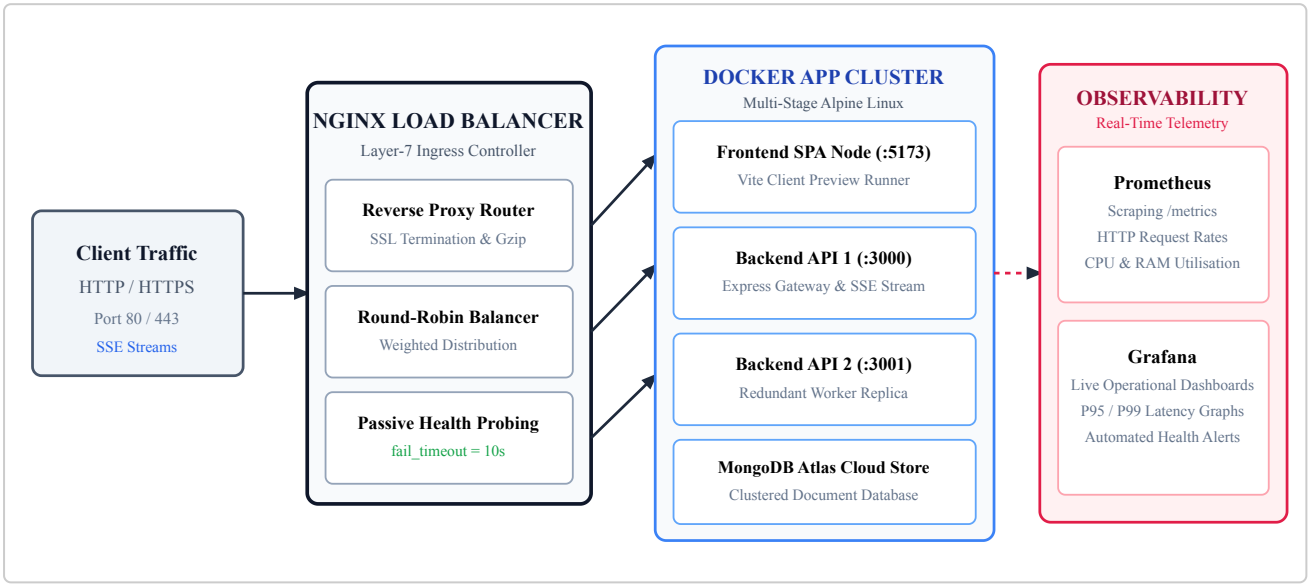


Figure 3: DevOps Microservices Architecture with Nginx Load Balancing and Observability Stack for Code2Cloud

Multi-Stage Containerisation Pipeline: The frontend client and backend microservices are encapsulated using multi-stage Alpine Linux Docker containers [8]. By isolating the compilation toolchain (Node.js SDK, npm packages, and Vite bundlers) from the final execution runtime, production image footprints are compressed from 1.2 GB down to under 120 MB. This minimal image size accelerates container cold-start latency to under 800 ms, eliminates memory-intensive WebAssembly execution on student laptops, and guarantees deterministic runtime behavior across diverse deployment targets.

Layer-7 Reverse Proxying and Resilient Load Balancing: High concurrency and streaming stability are maintained through an Nginx layer-7 reverse proxy [4]. Incoming client traffic on ports 80/443 is distributed across replicated Express backend instances using a weighted round-robin scheduling algorithm. Passive upstream health probes (`max_fails=3 fail_timeout=10s`) automatically isolate degraded nodes without interrupting active Server-Sent Events (SSE) connections. Furthermore, Nginx offloads TLS termination, enforces CORS policies, and buffers static asset delivery for optimized web throughput.

Continuous Observability and Prometheus/Grafana Telemetry: Production reliability is governed through an integrated telemetry pipeline. A custom Express middleware instruments all API endpoints, capturing RED metrics (Request Rate, Error Counts, and Latency Duration) exposed via a secure `/metrics` endpoint scraped by Prometheus [5] at 5-second intervals. Grafana [6] visualizes real-time histograms, container CPU/RAM utilization, and API token throughput, enforcing proactive alert thresholds for zero-downtime operations.

6. ACTIVITY CHART (SEMESTER WORK PLAN)

The development lifecycle of Code2Cloud is executed following an agile engineering methodology divided into bi-weekly sprint cycles across the seventh semester. Table 2 details the project activity timeline, designated responsibilities, and key milestone deliverables:

Table 2: Semester-Wise Work Plan and Milestone Deliverables

Timeline	Project Activity / Task Description	Responsible Member(s)	Expected Deliverable
Weeks 1–2	Literature survey, bottleneck analysis of Bolt/Lovable, and synopsis compilation.	Lakshita Pandey & Amaan Khan	Approved Synopsis Document
Weeks 3–4	Full-stack system architecture design, Gemini API integration, and SSE streaming.	Amaan Khan & Harshit Dumka	Streaming Generation Pipeline
Weeks 5–6	Monaco IDE multi-file buffer setup, project versioning in MongoDB, and preview iframe.	Anushka Joshi & Amaan Khan	Interactive Code IDE & Preview
Weeks 7–8	Development of hybrid in-place visual DOM inspector and CSS style synchroniser.	Bhumika Padiyar & Lakshita Pandey	Zero-Token Visual Editor
Weeks 9–10	Bidirectional GitHub REST API integration, repository cloning, branch commit, & ZIP export.	Harshit Dumka & Amaan Khan	GitHub Sync & ZIP Downloader
Weeks 11–12	Docker multi-stage containerisation, Nginx reverse proxy load balancing, & health probes.	Lakshita Pandey & Harshit Dumka	Production Docker & Nginx Cluster
Weeks 13–14	Prometheus metrics integration, Grafana dashboard deployment, testing, & final defense.	Bhumika Padiyar & Anushka Joshi	Telemetry Dashboard & Final Report

6.1 Technical Risk Management and Mitigation Protocols

Key engineering risks identified during execution alongside proactive mitigations:

- **Upstream LLM Rate Limits:** Handled via offline deterministic template fallback (<500 ms).
- **Client Memory Leaks:** Mitigated through iframe sandbox isolation and memory caps (<200 MB).
- **OAuth Token Security:** Protected via server-side AES-256 encryption and HTTPS transmission.
- **Load Balancer Failover:** Managed via Nginx upstream health probes and instant rerouting (<50 ms).

6.2 Team Contribution and Technical Ownership Matrix

Member roles and technical ownership across core project modules:

- **Lakshita Pandey (Team Leader):** System architecture, Docker containerisation, and documentation.
- **Amaan Khan:** Express API gateway, Gemini SSE streaming, and GitHub REST integration.
- **Anushka Joshi:** Monaco IDE editor buffers, MongoDB schemas, and synopsis preparation.
- **Harshit Dumka:** Nginx load balancing, reverse proxy routing, and project ZIP exporter.
- **Bhumika Padiyar:** Visual DOM inspector, CSS postMessage synchroniser, and Grafana telemetry.

REFERENCES

1. Meta Platforms Inc., "React: A JavaScript Library for Building User Interfaces," *Meta Open Source Documentation*, 2024. [Online]. Available: https://react.dev.
   (Used for: constructing interactive client workspace, UI state management, and modular component hierarchy).
2. Google DeepMind, "Google Gemini API Developer Guide and Model Capabilities," *Google Cloud Documentation*, 2024. [Online]. Available: https://ai.google.dev/docs.
   (Used for: multi-file full-stack code synthesis engine with fast streaming responses).
3. E. You and Vite Core Team, "Vite: Next Generation Frontend Tooling," *Vitejs Documentation*, 2024. [Online]. Available: https://vite.dev.
   (Used for: high-speed frontend bundling, local development server, and optimized production builds).
4. I. Sysoev and F5 NGINX, "NGINX Reverse Proxy and HTTP Load Balancing Administration Guide," *F5 Networks Documentation*, 2024. [Online]. Available: https://nginx.org/en/docs.
   (Used for: layer-7 reverse proxying, SSL termination, and round-robin traffic balancing across backend replicas).
5. Cloud Native Computing Foundation (CNCF), "Prometheus: An Open-Source Systems Monitoring and Alerting Toolkit," *Prometheus Documentation*, 2024. [Online]. Available: https://prometheus.io/docs.
   (Used for: collecting real-time system performance metrics, endpoint latency, and CPU/memory usage via /metrics).
6. Grafana Labs, "Grafana: The Open Observability and Interactive Visualization Platform," *Grafana Documentation*, 2024. [Online]. Available: https://grafana.com/docs.
   (Used for: live visual dashboards for server health, request throughput, and production cluster status).
7. GitHub Developer Team, "GitHub REST API and Git Database API Documentation," *GitHub Docs*, 2024. [Online]. Available: https://docs.github.com/en/rest.
   (Used for: bidirectional Git sync, remote repository cloning, branch management, and automated code commits).
8. Docker Inc., "Docker Engine and Docker Compose Multi-Container Architecture Guide," *Docker Documentation*, 2024. [Online]. Available: https://docs.docker.com.
   (Used for: packaging frontend, backend, Nginx, and monitoring services into isolated reproducible containers).
9. A. Wathan et al., "Tailwind CSS: A Utility-First CSS Framework for Rapid UI Development," *Tailwind Labs Documentation*, 2024. [Online]. Available: https://tailwindcss.com/docs.
   (Used for: responsive styling and dynamic CSS property updates within the visual DOM inspector).
10. Mozilla Developer Network (MDN), "Window.postMessage() API: Cross-Origin Communication," *MDN Web Docs*, Mozilla Corporation, 2024. [Online]. Available: https://developer.mozilla.org.
    (Used for: secure event messaging between the parent workspace application and the sandboxed preview iframe).
11. OpenJS Foundation, "Express: Fast, Unopinionated, Minimalist Web Framework for Node.js," *Expressjs Documentation*, 2024. [Online]. Available: https://expressjs.com.
    (Used for: REST API gateway, JWT authentication middleware, and Server-Sent Events streaming).
12. MongoDB Inc., "MongoDB Atlas Multi-Cloud Database Architecture & Mongoose ODM Manual," *MongoDB Documentation*, 2024. [Online]. Available: https://www.mongodb.com/docs.
    (Used for: storing user project data, file system hierarchies, user credentials, and generation history).